HDOC Day-41

****

Menu-driven C program to read two paragraphs and perform string operations using separate user-defined functions and pointer-based output arrays.

1. Problem Understanding

Read two paragraphs. Provide menu options to re-enter both paragraphs, find the longest and shortest words, remove repeated spaces, convert to title case, find case-insensitive word frequencies, and exit. Display results separately for both paragraphs.

2. Standard Test Cases

Test	Paragraph 1 input	Paragraph 2 input	Menu choices / further input	Expected result
TC1	hello hello world	THE quick FOX	Initial: both paragraphs as shown. Choices: 2, 3, 4, 5, 6 (Exit).	2: P1 longest hello/world; shortest hello/world. P2 longest quick; shortest THE/FOX. 3: P1 hello hello world; P2 THE quick FOX. 4: P1 Hello Hello World; P2 The Quick Fox. 5: P1 hello=2, world=1; P2 the=1, quick=1, fox=1.
TC2	a a A	Hi hi	Initial: both paragraphs as shown. Choices: 5, 6 (Exit).	5: P1 a=3; P2 hi=2 (case-insensitive).
TC3		one	Initial: P1 contains 3 spaces; P2 one. Choices: 2, 3, 5, 6 (Exit).	2: P1 no words; P2 longest=one, shortest=one. 3: P1 empty after space removal; P2 one. 5: P1 no words; P2 one=1.
TC4	red blue	green yellow	Initial: P1 red blue; P2 green yellow. Choice 1: new P1 cat dog; new P2 dog cat. Then choices: 5, 6 (Exit).	1: Both paragraphs replaced. 5: P1 cat=1, dog=1; P2 dog=1, cat=1.

3. Algorithm

1. Read both paragraphs with readParagraph() using fgets().
2. Display the menu repeatedly until choice 6.
3. For choice 1, read both paragraphs again.

4. For choice 2, copy each paragraph, split words with strtok(), and compare word lengths.

5. For choice 3, scan characters and copy only one space between words.

6. For choice 4, capitalize first letters and lowercase remaining letters of words.

7. For choice 5, lowercase words, count repeated words, and print frequencies.

8. For choice 6, exit; otherwise handle invalid choices.

4. C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
void readParagraph(char paragraph[], size_t size)
{
  fgets(paragraph, size, stdin);
  int i;
  for (i = 0; paragraph[i] != '\0'; i++)
  {
    if (paragraph[i] == '\n')
    {
      paragraph[i] = '\0';
      break;
    }
  }
}
void findLongestAndShortestWords(const char paragraph[], char longestWord[], char shortestWord[])
{
  char copy[1000];
  char *word;
  strcpy(copy, paragraph);
  longestWord[0] = '\0';
  shortestWord[0] = '\0';
  word = strtok(copy, " \t.,!?;:\n");
  while (word != NULL)
  {
    if (strlen(word) > strlen(longestWord))
```

```
        {
            strcpy(longestWord, word);
        }
        if (shortestWord[0] == '\0' || strlen(word) < strlen(shortestWord))
        {
            strcpy(shortestWord, word);
        }
        word = strtok(NULL, " \t.,!?;:\n");
    }
}
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[])
{
    int i, j = 0;
    for (i = 0; paragraph[i] != '\0'; i++)
    {
        if (isspace((unsigned char)paragraph[i]))
        {
            if (j > 0 && modifiedParagraph[j - 1] != ' ')
            {
                modifiedParagraph[j] = ' ';
                j++;
            }
        }
        else
        {
            modifiedParagraph[j] = paragraph[i];
            j++;
        }
    }
    if (j > 0 && modifiedParagraph[j - 1] == ' ')
    {
        j--;
    }
    modifiedParagraph[j] = '\0';
}
```

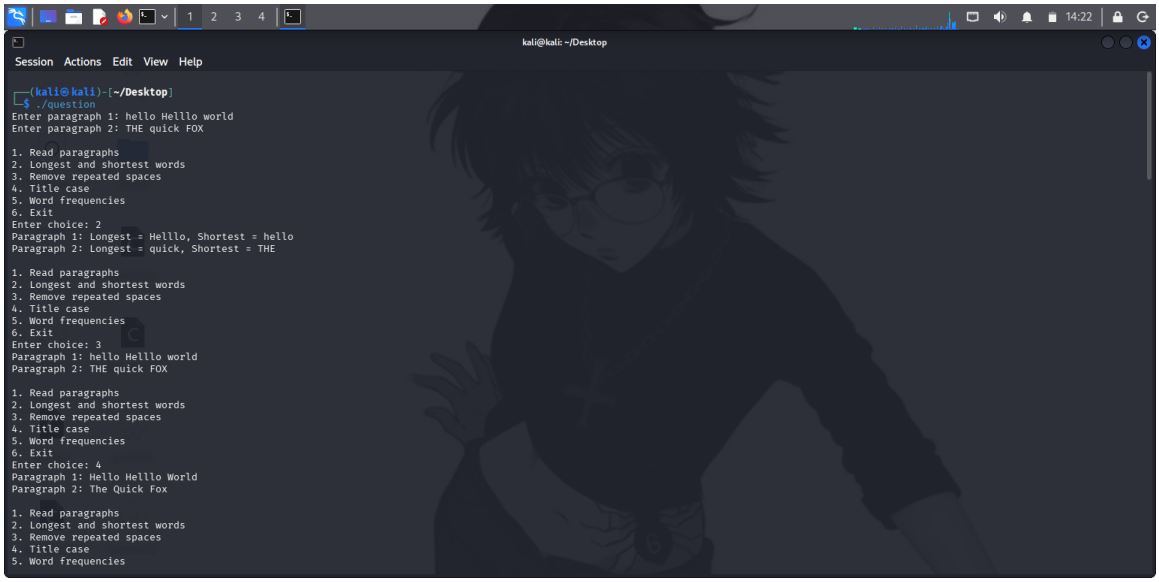
```
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[])
{
   int i, newWord = 1;
   for (i = 0; paragraph[i] != '\0'; i++)
   {
      if (isalpha((unsigned char)paragraph[i]))
      {
         if (newWord == 1)
         {
            titleCaseParagraph[i] = toupper((unsigned char)paragraph[i]);
         }
         else
         {
            titleCaseParagraph[i] = tolower((unsigned char)paragraph[i]);
         }
         newWord = 0;
      }
      else
      {
         titleCaseParagraph[i] = paragraph[i];
         if (isspace((unsigned char)paragraph[i]))
         {
            newWord = 1;
         }
      }
   }
   titleCaseParagraph[i] = '\0';
}
void findWordFrequencies(const char paragraph[])
{
   char copy[1000], words[150][100];
   int frequency[150] = {0};
   int count = 0, i, j, found;
   char *word;
   strcpy(copy, paragraph);
```

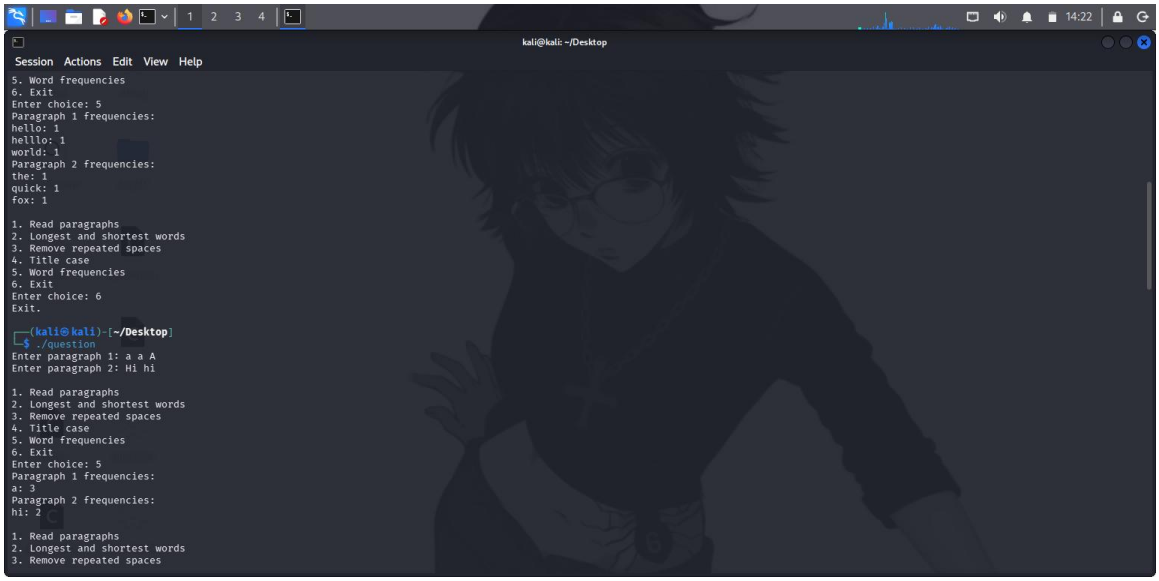
```
    word = strtok(copy, " \t.,!?;:\n");
    while (word != NULL)
    {
        for (i = 0; word[i] != '\0'; i++)
        {
            word[i] = tolower((unsigned char)word[i]);
        }
        found = 0;
        for (j = 0; j < count; j++)
        {
            if (strcmp(words[j], word) == 0)
            {
                frequency[j]++;
                found = 1;
                break;
            }
        }
        if (found == 0 && count < 150 && strlen(word) < 100)
        {
            strcpy(words[count], word);
            frequency[count] = 1;
            count++;
        }
        word = strtok(NULL, " \t.,!?;:\n");
    }
    if (count == 0)
    {
        printf("No words found.\n");
    }
    for (i = 0; i < count; i++)
    {
        printf("%s: %d\n", words[i], frequency[i]);
    }
}
int main()
```

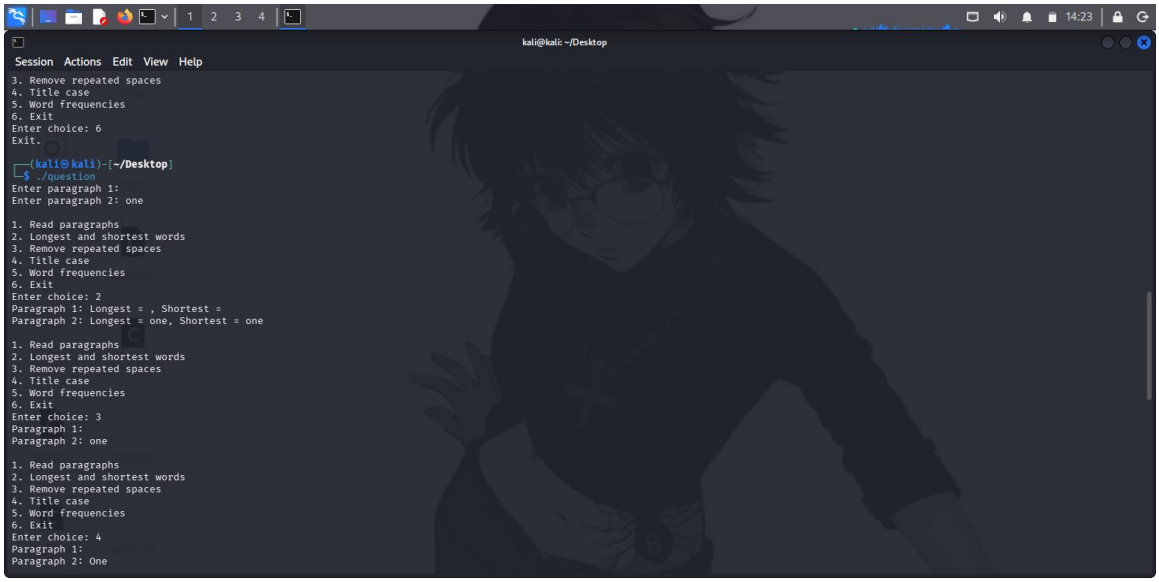
```
{
  char paragraph1[1000], paragraph2[1000];
  char longest[1000], shortest[1000];
  char modified[1000], title[1000];
  int choice;
  printf("Enter paragraph 1: ");
  readParagraph(paragraph1, sizeof(paragraph1));
  printf("Enter paragraph 2: ");
  readParagraph(paragraph2, sizeof(paragraph2));
  do
  {
    printf("\n1. Read paragraphs\n");
    printf("2. Longest and shortest words\n");
    printf("3. Remove repeated spaces\n");
    printf("4. Title case\n");
    printf("5. Word frequencies\n");
    printf("6. Exit\n");
    printf("Enter choice: ");
    if (scanf("%d", &choice) != 1)
    {
      printf("Invalid input.\n");
      return 1;
    }
    getchar();
    switch (choice)
    {
      case 1:
        printf("Enter paragraph 1: ");
        readParagraph(paragraph1, sizeof(paragraph1));
        printf("Enter paragraph 2: ");
        readParagraph(paragraph2, sizeof(paragraph2));
        break;
      case 2:
        findLongestAndShortestWords(paragraph1, longest, shortest);
        printf("Paragraph 1: Longest = %s, Shortest = %s\n", longest, shortest);
```

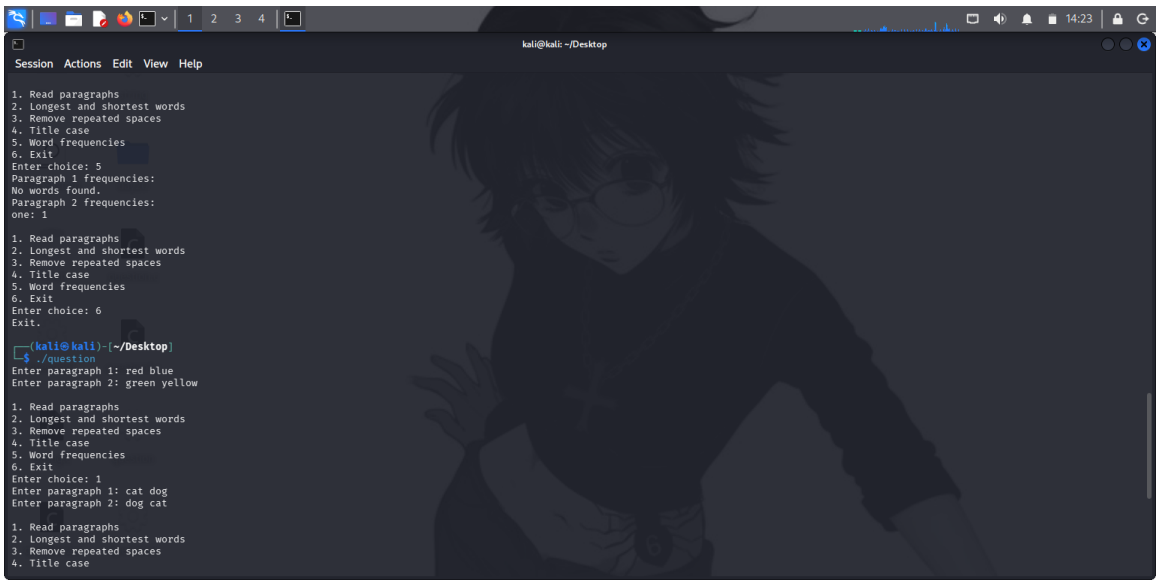
```
            findLongestAndShortestWords(paragraph2, longest, shortest);
            printf("Paragraph 2: Longest = %s, Shortest = %s\n", longest, shortest);
            break;
        case 3:
            removeRepeatedSpaces(paragraph1, modified);
            printf("Paragraph 1: %s\n", modified);
            removeRepeatedSpaces(paragraph2, modified);
            printf("Paragraph 2: %s\n", modified);
            break;
        case 4:
            convertToTitleCase(paragraph1, title);
            printf("Paragraph 1: %s\n", title);
            convertToTitleCase(paragraph2, title);
            printf("Paragraph 2: %s\n", title);
            break;
        case 5:
            printf("Paragraph 1 frequencies:\n");
            findWordFrequencies(paragraph1);
            printf("Paragraph 2 frequencies:\n");
            findWordFrequencies(paragraph2);
            break;
        case 6:
            printf("Exit.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }
  }
  while (choice != 6);
  return 0;
}
```

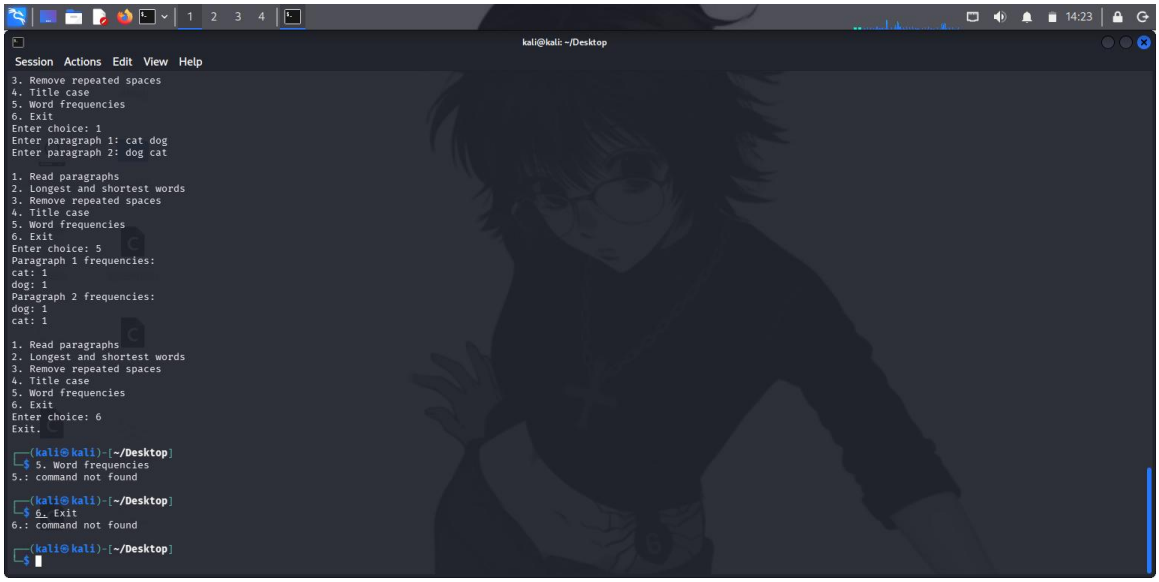
5. Compilation & Execution Evidence






kali@kali: ~/Desktop
Session Actions Edit View Help
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 6
Exit.
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question
Enter paragraph 1:
Enter paragraph 2: one
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 2
Paragraph 1: Longest = , Shortest =
Paragraph 2: Longest = one, Shortest = one
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 3
Paragraph 1:
Paragraph 2: one
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 4
Paragraph 1:
Paragraph 2: One


kali@kali: ~/Desktop
Session Actions Edit View Help
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 5
Paragraph 1 frequencies:
No words found.
Paragraph 2 frequencies:
one: 1
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 6
Exit.
┌──(kali㉿kali)-[~/Desktop]
└─$ ./question
Enter paragraph 1: red blue
Enter paragraph 2: green yellow
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case
5. Word frequencies
6. Exit
Enter choice: 1
Enter paragraph 1: cat dog
Enter paragraph 2: dog cat
1. Read paragraphs
2. Longest and shortest words
3. Remove repeated spaces
4. Title case



6. Verification of All Test Cases

Test	Input covered	Expected / verification	Status
TC1	P1: hello hello world P2: THE quick FOX Choices: 2,3,4,5,6	Word lengths, repeated spaces, title case and case-insensitive word frequencies as listed in standard test cases.	Expected
TC2	P1: a a A P2: Hi hi Choices: 5,6	P1 a=3; P2 hi=2.	Expected
TC3	P1: three spaces P2: one Choices: 2,3,5,6	P1 has no words; P2 one=1; cleaned P1 empty.	Expected
TC4	P1: red blue P2: green yellow Choice 1 re-entry: cat dog / dog cat Then 5,6	After re-entry, both paragraphs have cat=1 and dog=1.	Expected